

Week 11 Summer Homework: Fitting, Taylor Expansion, and Game Theory

Goal: understand fitting as optimization, then analyze a simple demand-response game.

AI policy for this homework. You may use AI or web search to learn syntax, understand official documentation, debug error messages, and ask conceptual questions. You may **not** ask AI to write the required MSE function, parameter-search loop, Taylor approximation code, Nash-equilibrium logic, or final explanation for you. Your submitted code must be your own manual programming work. If you use AI for explanation or debugging, write one sentence describing what it helped you understand.

Mathematical reminder. Fitting means choosing parameters for a predefined function so predictions are close to data. For a line,

$$\hat{y} = wx + b,$$

the parameters are slope w and intercept b . A common loss function is mean squared error:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2.$$

Game theory studies strategic choices. A pure-strategy Nash equilibrium is a cell in a payoff matrix where no player can improve by switching alone.

Monday: Translate Fitting Into an Optimization Problem

Use the small smart-grid load dataset below.

Hour x	Measured load y	Units
1	52	MW
2	56	MW
3	61	MW
4	65	MW
5	69	MW

Assume the predefined parametric function is a line $\hat{y} = wx + b$.

1. Define the input variable, output variable, and two parameters in words.
2. Write the prediction formula for each hour using w and b .
3. Write the MSE objective function for this dataset.
4. Explain in two sentences why finding the best line is an optimization problem.
5. Without using Python, compare the two candidate models $\hat{y} = 4x + 48$ and $\hat{y} = 3x + 51$ by calculating their squared errors and MSE values.

Tuesday: Complete the MSE Function and Parameter Search

Create a Python file named `week11_fitting_game.py`. Start from this skeleton and manually fill the TODO sections.

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([52, 56, 61, 65, 69], dtype=float)

def mse_line(w, b):
    # TODO: compute predictions y_hat = w*x + b
    # TODO: compute errors y - y_hat
    # TODO: return the mean squared error as one scalar number
    pass

candidate_w = [2, 3, 4, 5]
candidate_b = [45, 48, 51, 54]

best_w = None
best_b = None
```

```

best_mse = None

for w in candidate_w:
    for b in candidate_b:
        score = mse_line(w, b)
        # TODO: update best_w, best_b, and best_mse
        # if this score is the first score or the lowest score so far
        pass

print("best w:", best_w)
print("best b:", best_b)
print("best MSE:", best_mse)

```

1. Implement `mse_line(w, b)` without calling `sklearn.metrics`.
2. Implement the nested-loop search over candidate slopes and intercepts.
3. Print every candidate pair and its MSE before printing the best pair.
4. Explain why this search is a simple optimization algorithm even though it is not advanced.
5. Add one new candidate slope and one new candidate intercept, then report whether the best line changes.

Manual coding rule. Do not use AI to write the function or loop. You may use `np.mean`, array arithmetic, a manually written loop, or ordinary arithmetic.

Wednesday: Taylor Expansion as a Local Fit

Taylor expansion creates a simple local approximation to a function near one point. For this homework, use

$$f(x) = x^2 + 2x + 1.$$

1. Compute $f'(x)$ by hand.
2. At $a = 2$, calculate $f(a)$ and $f'(a)$.
3. Write the first-order Taylor approximation

$$T_1(x) = f(a) + f'(a)(x - a).$$

4. Use your approximation to estimate $f(2.1)$ and compare it with the exact value.
5. In two sentences, explain why Taylor expansion is a “local fit” instead of a global model.

Then add this function to your Python file.

```

def taylor_first_order(x_value, a):
    # f(x) = x^2 + 2x + 1
    # TODO: compute f(a)
    # TODO: compute f'(a)
    # TODO: return f(a) + f'(a) * (x_value - a)
    pass

print("Taylor estimate:", taylor_first_order(2.1, 2.0))
print("Exact value:", 2.1**2 + 2*2.1 + 1)

```

Thursday: Demand Response and Nash Equilibrium

Two households choose whether to reduce AC during a peak-load event. Their payoff matrices are below. Rows are Household A's actions. Columns are Household B's actions.

$$A = \begin{bmatrix} 3 & 1 \\ 4 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 4 \\ 1 & 0 \end{bmatrix}.$$

Row 0 means A reduces; row 1 means A does not reduce. Column 0 means B reduces; column 1 means B does not reduce.

1. By hand, identify A's best response in each column.

2. By hand, identify B's best response in each row.
3. Mark all pure-strategy Nash equilibria.
4. Explain whether the Nash equilibria are fair for both households.
5. Explain one incentive change that could make "both reduce" more attractive.

Add this code skeleton to your Python file and complete the TODO sections.

```
A = np.array([[3, 1],
              [4, 0]], dtype=float)
B = np.array([[3, 4],
              [1, 0]], dtype=float)

action_A = ["reduce", "do not reduce"]
action_B = ["reduce", "do not reduce"]

for i in range(2):
    for j in range(2):
        # TODO: check whether A[i, j] is A's best payoff
        # when B's action j is fixed
        a_best = None

        # TODO: check whether B[i, j] is B's best payoff
        # when A's action i is fixed
        b_best = None

        if a_best and b_best:
            print("Nash equilibrium:", action_A[i], "/", action_B[j])
```

Hint. When B's action is fixed, A compares values within one column of matrix A . When A's action is fixed, B compares values within one row of matrix B .

Friday: Feynman Technique Post

Write a beginner-friendly post explaining one of the two topics below.

1. **Option A: Fitting as optimization.** Explain data points, a predefined parametric function, parameters, prediction error, MSE, and why the best-fit line minimizes a loss function.
2. **Option B: Nash equilibrium in demand response.** Explain players, actions, payoffs, best responses, Nash equilibrium, and why a stable outcome is not always the fairest or socially best outcome.

Your post must include:

1. one smart-grid example,
2. one small table, diagram, screenshot, or hand-drawn picture,
3. one sentence connecting the topic back to optimization,
4. one question you still have after this week.

Submission checklist: Monday hand calculations; Tuesday `mse_line` and parameter search; Wednesday Taylor approximation by hand and in code; Thursday Nash-equilibrium analysis by hand and in code; Friday Feynman note.